Project Report

Subject Name: Python Programming

Subject Code: 24CAH-301

Project Name: WEATHER DATA ANALYSIS SYSTEM

Submitted by:	Submitted To:
Anshuman	Ms. Jyoti Rani
24BCA10063	
Section: 24BCA – 1 'A'	

WEATHER DATA ANALYSIS SYSTEM

Mini Project Report — Station Data Simulation, Quality Control, Climate Statistics, Trend Forecasting & Weather Warnings

Project Type: Academic Mini Project **Primary Language:** Python 3.x (Standard Library only) **IDE:** Visual Studio Code **Status:** Tested — full program run completed

1. Abstract

Weather stations do not always report correct numbers. A damaged sensor, a loose cable or a transmission glitch can produce a temperature of -80 °C or a wind speed of 400 km/h, and one such value is enough to distort an average or trigger a false warning. The Weather Data Analysis System is a Python program built around this problem: it gathers observations from five stations, checks every reading before using it, and only then calculates statistics, comfort indices, a short forecast and warnings.

The program is organised as a pipeline of six parts: (1) a data model for a single observation, (2) a simulator that generates ten days of readings at six-hour intervals, (3) a two-layer quality-control step made of range checks and a z-score outlier test, (4) derived indices such as dew point, wind chill and the Thom discomfort index, (5) a forecasting routine based on a least-squares trend line, and (6) a warning module together with CSV and JSON export. No third-party library is required; everything uses modules that ship with Python.

2. Project Design & Implementation

Unlike a purely class-based layout, this project uses a Python dataclass for the data record and small, single-purpose functions for each processing step. This keeps every step easy to test on its own and easy to replace later, for instance swapping the simulator for a real data feed.

Step	Component	Function / Class	Responsibility
1	Data model	Observation (dataclass)	Holds one reading and a list of quality issues
2	Collection	simulate()	Creates 200 readings: 5 stations × 10 days × 4 per day
3	Quality control	quality_control()	Range check, then per-station z-score outlier check
4	Indices	dew_point(), wind_chill(), discomfort_index()	Derived comfort and moisture measures
5	Forecast	linear_fit(), forecast()	Least-squares trend over the last 12 readings
6	Warnings / Export	raise_alerts(), main()	Threshold rules, console report, CSV and JSON files

2.1 Monitoring Stations

Five stations with clearly different conditions are simulated. Each station has a mean temperature and mean humidity; a 24-hour cosine curve (warmest at 3 PM), a slow warming drift and Gaussian noise are added to create believable readings. Pressure is generated around 1008 hPa with a station-specific offset.

Station	Mean Temp (°C)	Mean Humidity (%)	Character
Patiala	28.0	54	Plains, warm
Mohali	27.0	57	Plains, moderate
Bathinda	29.5	48	Dry, hot south-west Punjab
Dehradun	24.0	66	Foothill valley, humid
Manali	12.0	72	Mountain, cold and humid

2.2 Quality Control

Quality control happens in two layers. Layer one rejects any value outside what is physically possible. Layer two looks at the remaining temperatures of each station and flags a reading if it lies more than 3.5 standard deviations from that station's mean. A rejected reading is marked with an issue label, kept out of every calculation, and the program carries on without stopping.

Quantity	Allowed minimum	Allowed maximum
Temperature	-40 °C	50 °C
Humidity	0 %	100 %
Pressure	870 hPa	1085 hPa
Wind speed	0 km/h	200 km/h
Rainfall	0 mm	250 mm

To prove that this stage works, four faulty values are inserted on purpose: a temperature of -80 °C (Bathinda), a humidity of -15 % (Manali), a wind speed of 410 km/h (Patiala) and a pressure of 1500 hPa (Dehradun).

2.3 Mathematical Formulations

- **Dew point (Magnus formula):** with a = 17.625 and b = 243.04, let γ = ln(RH/100) + aT/(b+T); then Td = bγ/(a−γ).
- **Wind chill (Environment Canada):** WC = 13.12 + 0.6215T − 11.37V^0.16 + 0.3965T·V^0.16, applied only when T ≤ 10 °C and V ≥ 4.8 km/h.
- **Thom discomfort index:** DI = T − 0.55(1 − 0.01·RH)(T − 14.5). Values below 21 are comfortable, 21–24 slightly warm, 24–27 uncomfortable and 27 or more very uncomfortable.
- **Z-score:** z = |x − μ| / σ, used to detect abnormal temperatures inside the allowed range.
- **Trend forecast:** a straight line y = a + b·x is fitted by least squares to the last 12 temperatures, with slope b = Σ(x−x̄)(y−ȳ) / Σ(x−x̄)². The line is extended four steps to give +6 h, +12 h, +18 h and +24 h values.

2.4 Warning Rules

Warnings are evaluated only for the most recent observation of every station.

Warning	Condition
HEAT STRESS	Discomfort index ≥ 27
INTENSE RAIN	Rainfall ≥ 10 mm
STRONG WIND	Wind speed ≥ 25 km/h
COLD STRESS	Wind chill ≤ 8 °C
PRESSURE DROP	Pressure at least 4 hPa below the station's own average

3. Python Code

The complete program is saved as weather_analysis.py. It contains the data model, simulator, quality control, indices, forecasting, warnings and file export:

```
"""Weather Data Analysis System - dataclass / function based design."""
import csv
import json
import math
import random
import statistics
from dataclasses import dataclass, field, asdict
from datetime import datetime, timedelta
from typing import Dict, List, Tuple

random.seed(2024)

# ---------------------------------------------------------------
# 1. DATA MODEL
# ---------------------------------------------------------------
@dataclass
class Observation:
    station: str
    time: datetime
    temp: float          # degree Celsius
    humidity: float      # percent
    pressure: float      # hPa
    wind: float          # km/h
    rain: float          # mm
    issues: List[str] = field(default_factory=list)

    @property
    def ok(self) -> bool:
        return not self.issues

    def as_row(self) -> Dict:
        d = asdict(self)
        d["time"] = self.time.strftime("%Y-%m-%d %H:%M")
        d["issues"] = "|".join(self.issues)
        for k in ("temp", "humidity", "pressure", "wind", "rain"):
            d[k] = round(d[k], 2)
        return d


# ---------------------------------------------------------------
# 2. SENSOR SIMULATION
```

```
# ---------------------------------------------------------------
STATIONS = {
    #  name        (mean temp, mean humidity, altitude effect on pressure)
    "Patiala":     (28.0, 54, 0),
    "Mohali":      (27.0, 57, 0),
    "Bathinda":    (29.5, 48, 2),
    "Dehradun":    (24.0, 66, -12),
    "Manali":      (12.0, 72, -90),
}

def simulate(days: int = 10, step_hours: int = 6) -> List[Observation]:
    t0 = datetime(2026, 10, 18, 0, 0)
    data = []
    steps = days * 24 // step_hours
    for i in range(steps):
        ts = t0 + timedelta(hours=i * step_hours)
        phase = math.cos((ts.hour - 15) * 2 * math.pi / 24)   # peak at 3 PM
        drift = 0.08 * i / 4                                   # slow warming trend
        for name, (tm, hm, dp) in STATIONS.items():
            temp = tm + 5.5 * phase + drift + random.gauss(0, 0.9)
            hum = min(99, max(8, hm - 14 * phase + random.gauss(0, 3)))
            pres = 1008 + dp + random.gauss(0, 2.2)
            wind = abs(8 + 4 * phase + random.gauss(0, 3))
            rain = random.choices([0, 0.8, 3.5, 11.0], [70, 14, 11, 5])[0]
            data.append(Observation(name, ts, temp, hum, pres, wind, rain))

    # deliberately corrupted readings to test the checker
    for o in data:
        if o.station == "Bathinda" and o.time == datetime(2026, 10, 20, 12):
            o.temp = -80.0
        if o.station == "Manali" and o.time == datetime(2026, 10, 23, 18):
            o.humidity = -15.0
        if o.station == "Patiala" and o.time == datetime(2026, 10, 25, 6):
            o.wind = 410.0
        if o.station == "Dehradun" and o.time == datetime(2026, 10, 22, 0):
            o.pressure = 1500.0
    return data


# ---------------------------------------------------------------
# 3. QUALITY CONTROL  (range check + z-score outlier check)
# ---------------------------------------------------------------
RANGES = {"temp": (-40, 50), "humidity": (0, 100), "pressure": (870, 1085),
          "wind": (0, 200), "rain": (0, 250)}

def quality_control(data: List[Observation], z_limit: float = 3.5
                    ) -> Tuple[List[Observation], Dict[str, int]]:
    for o in data:
        for name, (lo, hi) in RANGES.items():
            if not lo <= getattr(o, name) <= hi:
                o.issues.append(f"{name}_range")

    # second pass: statistical outliers inside the allowed range, per station
    by_station: Dict[str, List[Observation]] = {}
    for o in data:
        by_station.setdefault(o.station, []).append(o)
    for obs in by_station.values():
        good = [o.temp for o in obs if o.ok]
        mu, sd = statistics.mean(good), statistics.pstdev(good)
        for o in obs:
            if o.ok and sd and abs(o.temp - mu) / sd > z_limit:
```

```
                o.issues.append("temp_zscore")

    clean = [o for o in data if o.ok]
    report = {"total": len(data), "accepted": len(clean),
              "rejected": len(data) - len(clean)}
    return clean, report


# ---------------------------------------------------------------
# 4. DERIVED INDICES
# ---------------------------------------------------------------
def dew_point(t: float, rh: float) -> float:
    a, b = 17.625, 243.04                       # Magnus constants
    g = math.log(max(rh, 1) / 100) + a * t / (b + t)
    return b * g / (a - g)

def wind_chill(t: float, v: float) -> float:
    if t > 10 or v < 4.8:
        return t
    return 13.12 + 0.6215 * t - 11.37 * v ** 0.16 + 0.3965 * t * v ** 0.16

def discomfort_index(t: float, rh: float) -> float:
    """Thom's discomfort index."""
    return t - 0.55 * (1 - 0.01 * rh) * (t - 14.5)

def comfort_label(di: float) -> str:
    if di < 21: return "Comfortable"
    if di < 24: return "Slightly warm"
    if di < 27: return "Uncomfortable"
    return "Very uncomfortable"

def linear_fit(y: List[float]) -> Tuple[float, float]:
    n = len(y)
    xs = range(n)
    xm, ym = (n - 1) / 2, sum(y) / n
    sxx = sum((x - xm) ** 2 for x in xs)
    slope = sum((x - xm) * (v - ym) for x, v in zip(xs, y)) / sxx
    return slope, ym - slope * xm

def forecast(temps: List[float], horizon: int = 4, window: int = 12) -> List[float]:
    """Least-squares line over the last `window` readings, extended forward."""
    seg = temps[-window:]
    slope, icpt = linear_fit(seg)
    n = len(seg)
    return [round(icpt + slope * (n + k), 2) for k in range(horizon)]


# ---------------------------------------------------------------
# 5. STATION SUMMARY AND ALERTS
# ---------------------------------------------------------------
def summarise(clean: List[Observation]) -> Dict[str, Dict]:
    out = {}
    for name in STATIONS:
        obs = [o for o in clean if o.station == name]
        t = [o.temp for o in obs]
        di = [discomfort_index(o.temp, o.humidity) for o in obs]
        out[name] = {
            "n": len(obs),
            "mean_temp": round(statistics.mean(t), 2),
            "median_temp": round(statistics.median(t), 2),
            "min_temp": round(min(t), 2),
```

```
            "max_temp": round(max(t), 2),
            "temp_sd": round(statistics.pstdev(t), 2),
            "mean_rh": round(statistics.mean(o.humidity for o in obs), 1),
            "mean_dew": round(statistics.mean(dew_point(o.temp, o.humidity) for o in obs), 2),
            "peak_di": round(max(di), 2),
            "rain_total": round(sum(o.rain for o in obs), 1),
            "rainy_obs": sum(1 for o in obs if o.rain > 0),
            "wind_max": round(max(o.wind for o in obs), 1),
        }
    return out

def raise_alerts(clean: List[Observation]) -> List[str]:
    last = max(o.time for o in clean)
    msgs = []
    avg_p = {n: statistics.mean(o.pressure for o in clean if o.station == n)
             for n in STATIONS}
    for o in clean:
        if o.time != last:
            continue
        di = discomfort_index(o.temp, o.humidity)
        wc = wind_chill(o.temp, o.wind)
        if di >= 27:
            msgs.append(f"HEAT STRESS | {o.station}: DI {di:.1f}")
        if o.rain >= 10:
            msgs.append(f"INTENSE RAIN | {o.station}: {o.rain:.1f} mm")
        if o.wind >= 25:
            msgs.append(f"STRONG WIND | {o.station}: {o.wind:.1f} km/h")
        if wc <= 8:
            msgs.append(f"COLD STRESS | {o.station}: wind chill {wc:.1f} C")
        drop = avg_p[o.station] - o.pressure
        if drop >= 4:
            msgs.append(f"PRESSURE DROP | {o.station}: {drop:.1f} hPa below station mean")
    return msgs


# ---------------------------------------------------------------
# 6. MAIN PROGRAM
# ---------------------------------------------------------------
def main():
    bar = "#" * 80
    print(bar); print("WEATHER DATA ANALYSIS SYSTEM".center(80)); print(bar)

    print("Step 1: simulating station data ...")
    raw = simulate()
    print(f"   {len(raw)} observations from {len(STATIONS)} stations")

    print("Step 2: running quality control ...")
    clean, qc = quality_control(raw)
    print(f"   accepted={qc['accepted']}  rejected={qc['rejected']}")
    for o in raw:
        if not o.ok:
            print(f"   - {o.station:<9} {o.time:%d %b %H:%M}  {', '.join(o.issues)}")

    print("Step 3: computing statistics and indices ...")
    summary = summarise(clean)

    print("Step 4: forecasting and alert checks ...")
    alerts = raise_alerts(clean)

    print("\n" + bar)
```

```
    hdr = f"{'Station':<10}{'Mean':>7}{'Median':>8}{'Min':>7}{'Max':>7}{'SD':>6}{'RH%':>7}{'Dew':>7}
{'PeakDI':>8}{'Rain':>9}"
    print(hdr); print("-" * len(hdr))
    for s, m in summary.items():
        print(f"{s:<10}{m['mean_temp']:>7.1f}{m['median_temp']:>8.1f}{m['min_temp']:>7.1f}"
              f"{m['max_temp']:>7.1f}{m['temp_sd']:>6.2f}{m['mean_rh']:>7.1f}"
              f"{m['mean_dew']:>7.1f}{m['peak_di']:>8.1f}{m['rain_total']:>7.1f}mm")
    print("-" * len(hdr))

    print("\nComfort at latest reading:")
    last = max(o.time for o in clean)
    for o in clean:
        if o.time == last:
            di = discomfort_index(o.temp, o.humidity)
            print(f"   {o.station:<9} T={o.temp:5.1f} C  RH={o.humidity:4.1f}%  DI={di:4.1f}  ->
{comfort_label(di)}")

    print("\nForecast (linear trend, next 4 readings = +6h, +12h, +18h, +24h):")
    forecasts = {}
    for s in STATIONS:
        temps = [o.temp for o in clean if o.station == s]
        forecasts[s] = forecast(temps)
        print(f"   {s:<9} last={temps[-1]:5.2f}  ->  " + " | ".join(f"{v:5.2f}" for v in forecasts[s]))

    print(f"\nALERTS ({len(alerts)}):")
    for a in alerts or ["No alerts for the latest observation."]:
        print("   * " + a)
    print(bar)

    with open("clean_observations.csv", "w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=list(clean[0].as_row()))
        w.writeheader()
        w.writerows(o.as_row() for o in clean)
    with open("analysis_report.json", "w", encoding="utf-8") as f:
        json.dump({"generated": datetime.now().isoformat(timespec="seconds"),
                   "quality_control": qc, "summary": summary,
                   "forecast": forecasts, "alerts": alerts}, f, indent=2)
    print("Saved clean_observations.csv and analysis_report.json")

if __name__ == "__main__":
    main()
```

4. Output / Screenshots

The program was executed with python weather_analysis.py. Because the random generator is seeded, every run prints the same results:

```
################################################################################
                          WEATHER DATA ANALYSIS SYSTEM
################################################################################
Step 1: simulating station data ...
   200 observations from 5 stations
Step 2: running quality control ...
   accepted=196  rejected=4
   - Bathinda  20 Oct 12:00  temp_range
   - Dehradun  22 Oct 00:00  pressure_range
   - Manali    23 Oct 18:00  humidity_range
   - Patiala   25 Oct 06:00  wind_range
Step 3: computing statistics and indices ...
Step 4: forecasting and alert checks ...

################################################################################
Station      Mean  Median    Min    Max    SD    RH%    Dew  PeakDI     Rain
--------------------------------------------------------------------------------
Patiala      28.5    30.0   22.7   33.5  3.93   54.2   18.0    27.9   35.2mm
Mohali       27.4    27.9   20.7   32.7  4.02   56.7   17.7    27.6   49.7mm
Bathinda     29.9    27.4   23.2   35.4  4.05   48.5   17.4    28.3   23.6mm
Dehradun     24.7    26.7   18.7   30.8  4.10   65.9   17.7    26.7   41.1mm
Manali       12.6    10.9    5.5   18.2  4.12   72.8    7.6    17.4   11.8mm
--------------------------------------------------------------------------------

Comfort at latest reading:
   Patiala   T= 30.5 C  RH=44.5%  DI=25.6  -> Uncomfortable
   Mohali    T= 32.3 C  RH=37.4%  DI=26.2  -> Uncomfortable
   Bathinda  T= 35.0 C  RH=39.0%  DI=28.1  -> Very uncomfortable
   Dehradun  T= 28.1 C  RH=60.0%  DI=25.1  -> Uncomfortable
   Manali    T= 14.9 C  RH=63.3%  DI=14.8  -> Comfortable

Forecast (linear trend, next 4 readings = +6h, +12h, +18h, +24h):
   Patiala   last=30.45  ->  29.71 | 29.75 | 29.78 | 29.82
   Mohali    last=32.29  ->  30.21 | 30.54 | 30.88 | 31.21
   Bathinda  last=34.95  ->  32.59 | 32.95 | 33.30 | 33.65
   Dehradun  last=28.12  ->  26.40 | 26.65 | 26.90 | 27.14
   Manali    last=14.90  ->  14.95 | 15.22 | 15.50 | 15.77

ALERTS (1):
   * HEAT STRESS | Bathinda: DI 28.1
################################################################################
Saved clean_observations.csv and analysis_report.json
```

4.1 Observations from the Output

- Out of 200 readings, 196 were accepted and 4 were rejected. The rejected ones are exactly the four faults inserted for testing, so the range checks behave as intended.
- The z-score layer rejected nothing extra, which is the correct result because the remaining data contain no hidden outliers.
- Bathinda has the highest mean temperature (29.9 °C) and Manali the lowest (12.6 °C), matching the base values chosen for the stations.
- Manali's average dew point (7.6 °C) is far below the other stations, which are all close to 17–18 °C.

- Mohali received the most rainfall (49.7 mm) and Manali the least (11.8 mm) in the ten-day window.
- Only one warning was raised for the final observation: HEAT STRESS for Bathinda, with a discomfort index of 28.1.
- The linear-trend forecast moves back toward each station's typical level, for example Bathinda from 34.95 °C to about 32.6 °C six hours later.

4.2 Exported Data Files

- **clean_observations.csv:** 196 validated rows with station, time, temp, humidity, pressure, wind, rain and issues.
- **analysis_report.json:** generation time, quality-control counts, per-station statistics, forecasts and warnings in machine-readable form.

5. Conclusion

The project demonstrates that a complete data-processing workflow can be written in plain Python. Checking the data first, using both fixed limits and a statistical test, keeps bad sensor values out of every later result. The program then turns the clean data into summaries, comfort ratings, a 24-hour trend forecast and automatic warnings. In future the simulator could be replaced by a live weather API, the straight-line forecast by seasonal or exponential-smoothing models, and charts could be added to visualise temperature and rainfall.

6. GitHub Repository Link

Project Repository: https://github.com/ash6158/Weather-Data-Analysis-System